# Bitrix24 Integration

## 1. Overview

### 1.1 Description

**Bitrix24 Integration** connects **Bitrix24** (CRM and business management platform) with **Newo Platform**.

It enables:

- Checking calendar availability for appointments
- Creating bookings as calendar events with contact linking
- Cancelling appointments by deleting calendar events
- Automatic deal creation at conversation end with employee load balancing
- Creating leads in Bitrix24 CRM
- Creating tasks with LLM-powered payload extraction
- Contact lookup and auto-creation across all flows
- Employee workload balancing for deal assignment

This integration is designed for **service businesses and teams** who use Bitrix24 as their CRM and need **automated scheduling, lead management, and task creation through a conversational AI agent** (voice or chat).

### 1.2 Use Cases

- When a client asks about availability → Query Bitrix24 calendar events and compute free slots
- When a client wants to book → Find or create contact, create calendar event
- When a client wants to cancel → Delete calendar event from Bitrix24
- When a conversation ends → Auto-create deal assigned to the least busy employee
- When a lead arrives (e.g., from Yelp) → Create lead in Bitrix24 CRM
- When a client requests a task → LLM extracts details, creates task in Bitrix24

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | ✅ | Calendar-based slot computation with working hours |
| Book Appointment | ✅ | Calendar event with CRM contact link |
| Cancel Appointment | ✅ | Deletes calendar event |
| Deal Creation | ✅ | Auto on session end with employee load balancing |
| Lead Creation | ✅ | From Yelp leads or webhook |
| Task Creation | ✅ | LLM-powered payload extraction from conversation |
| Contact Management | ✅ | Lookup by phone, auto-create if missing |
| Employee Load Balancing | ✅ | Assigns deals to employee with fewest open deals |
| Working Hours Detection | ✅ | Fetched from Bitrix24 calendar settings |
| Availability Preload | ✅ | Optional check on conversation start |

| | | |
|---|---|---|
| Reschedule | ❌ | Cancel + rebook as workaround |
| Multi-Calendar Support | ❌ | Single owner calendar per integration |

## 2. Requirements

Before installation:

- Active **Bitrix24** account (cloud or on-premise)
- **Bitrix24 OAuth App** with Client ID and Client Secret
- Calendar configured for the owner user (typically admin, ID=1)

## 3. How to Setup

### 3.1 Step 1 — Authorize in Bitrix24

1. In Newo platform, set `bitrix_base_url`, `bitrix_client_id`, and `bitrix_client_secret`
2. Click the Bitrix24 authorization link provided by the platform
3. Log in to your Bitrix24 account
4. Authorize the Newo app
5. Copy the **authorization code** from the redirect URL
6. Paste the code into the `bitrix_oauth_code` attribute

### 3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `bitrix_base_url` | ✅ | Bitrix24 instance URL (e.g., `https://yourcompany.bitrix24.com`) |
| `bitrix_client_id` | ✅ | OAuth App ID |
| `bitrix_client_secret` | ✅ | OAuth App Secret |
| `bitrix_oauth_code` | ✅ | One-time OAuth authorization code |
| `bitrix_owner_id` | ❌ | Calendar owner ID (default: `1`) |
| `bitrix_duration` | ❌ | Slot duration in minutes (default: `60`) |
| `bitrix_availability_days` | ❌ | Days ahead for availability (default: `1`) |
| `bitrix_employee_position_name` | ❌ | Employee position filter for deal assignment (default: `cleaner`) |
| `bitrix_create_deal_on_conversation_end` | ❌ | Auto-create deal on session end (default: `True`) |
| `bitrix_check_availability_on_conversation_start` | ❌ | Preload availability on start (default: `False`) |
| `bitrix_enable_slot_check` | ❌ | Enable availability checking (default: `True`) |

| | | |
|---|---|---|
| `bitrix_enable_booking` | ❌ | Enable booking (default: `True`) |
| `bitrix_enable_cancellation` | ❌ | Enable cancellation (default: `True`) |
| `bitrix_enable_task_creation` | ❌ | Enable task creation (default: `True`) |
| `bitrix_enable_lead_creation` | ❌ | Enable lead creation (default: `True`) |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Exchanges the OAuth code for access and refresh tokens
    - Fetches working hours from Bitrix24 calendar settings
    - Creates HTTP connectors for API and token operations
    - Registers the task creation tool with the agent
    - Injects booking and availability payload schemas

# 4. How to Use

## 4.1 How the Integration Works

The integration uses Bitrix24's **REST API** for calendar events, contacts, deals, leads, tasks, and user management. Availability is computed from calendar events. Deals are auto-created at session end with intelligent employee assignment.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Bitrix24

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Check Availability** | `get_available_slots` | When the agent needs to look up open slots |
| **Book Appointment** | `book_slot` | When the agent confirms a booking |
| **Cancel Appointment** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Session Ended** | `session_ended` | Auto-create deal on conversation end |
| **Create Task** | `create_task` | When agent creates a task from conversation |
| **Create Lead** | `yelp_new_lead` | When a new lead arrives (e.g., from Yelp) |
| **Conversation Start** | `conversation_started` | Optional availability preload |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |

**Available Actions**

- **Get Calendar Events** — Fetch events by date range (GET `/rest/calendar.event.get`)
- **Create Calendar Event** — New appointment (POST `/rest/calendar.event.add`)
- **Delete Calendar Event** — Cancel appointment (POST `/rest/calendar.event.delete`)
- **Search Contacts** — Find by phone (POST `/rest/crm.contact.list`)
- **Create Contact** — New CRM contact (POST `/rest/crm.contact.add`)
- **Create Deal** — New CRM deal with employee assignment (POST `/rest/crm.deal.add`)

- **Create Lead** — New CRM lead (POST `/rest/crm.lead.add`)
- **Create Task** — New task with contact link (POST `/rest/tasks.task.add`)
- **List Users** — Get employees by position (POST `/rest/user.get`)
- **List Deals** — Get deals for workload counting (POST `/rest/crm.deal.list`)

### 4.2 Architecture Diagrams

#### Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Bitrix24 Integration
    participant T as Bitrix24 OAuth
    participant API as Bitrix24 API

    Note over I,T: Setup (on publish)
    A->>I: project_publish_finish
    I->>T: GET /oauth/token/ (code)
    T-->>I: access_token + refresh_token
    I->>API: GET /rest/calendar.settings.get
    API-->>I: Working hours
    I->>A: Tools registered

    Note over U,API: Availability Check
    U->>A: "Any slots tomorrow?"
    A->>I: get_available_slots (date)
    I->>API: GET /rest/calendar.event.get (owner, date range)
    API-->>I: Calendar events
    I->>I: Compute free slots (subtract booked, slice by duration)
    I->>A: result_success (availability[])

    Note over U,API: Booking
    U->>A: "Book 2:00 PM"
    A->>I: book_slot (customerInfo, dateTime)
    I->>API: POST /rest/crm.contact.list (phone filter)
    alt Contact not found
        I->>API: POST /rest/crm.contact.add (name, phone)
    end
    I->>API: POST /rest/calendar.event.add (type, name, from, to, crm_fields)
    API-->>I: event_id
    I->>A: result_success (booking_id)

    Note over U,API: Session End -> Deal
    A->>I: session_ended
    I->>API: POST /rest/crm.contact.list (phone)
    I->>API: POST /rest/user.get (position filter)
    API-->>I: Employee list
    I->>API: POST /rest/crm.deal.list (stage=NEW)
    API-->>I: Deal counts per employee
    I->>I: Select least busy employee
    I->>API: POST /rest/crm.deal.add (title, contact, employee)
```

```
    API-->>I: deal_id

    Note over U,API: Task Creation
    U->>A: "Create a task for..."
    A->>I: create_task (memory)
    I->>I: LLM extract: name, phone, taskTitle
    I->>API: POST /rest/crm.contact.list (phone)
    I->>API: POST /rest/tasks.task.add (title, deadline, contact)
    API-->>I: task_id
    I->>A: result_success
```

**AvailabilityFlow**

```
flowchart TD
    E["get_available_slots<br>or conversation_started"] --> GATE{{"Slot
check<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| API["GET /rest/calendar.event.get<br>{ownerId, from, to}"]
    API --> PARSE["Extract booked events<br>with start/end times"]
    PARSE --> CALC["Sliding window:<br>generate slots at<br>duration intervals<br>
(default: 60 min)<br>skip overlapping"]
    CALC --> FORMAT["Group by date<br>Format as HH:MM AM/PM"]
    FORMAT --> OUT["result_success<br>{availability[]}"]
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> GATE{{"Booking<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| CONTACT["POST /rest/crm.contact.list<br>{phone filter}"]
    CONTACT --> FOUND{{"Contact<br>found?"}}
    FOUND -->|"No"| CREATE["POST /rest/crm.contact.add<br>{name, phone}"]
    CREATE --> CID["contact_id"]
    FOUND -->|"Yes"| CID
    CID --> EVENT["POST /rest/calendar.event.add<br>{type: user,<br>name: Client
Appointment,<br>from/to ISO times,<br>crm_fields: [C_contact_id]}"]
    EVENT --> OUT["result_success<br>{booking_id}"]
```

**DealFinalizationFlow (on session end)**

```
flowchart TD
    E["session_ended"] --> GATE{{"Deal creation<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| BOOK{{"Has bookings<br>in persona?"}}
    BOOK -->|"No"| SKIP
    BOOK -->|"Yes"| CONTACT["POST /rest/crm.contact.list<br>{phone}"]
    CONTACT --> EMP["POST /rest/user.get<br>{ACTIVE: Y,<br>position filter}"]
    EMP --> DEALS["POST /rest/crm.deal.list<br>{stage: NEW}"]
    DEALS --> BEST["Count deals per employee<br>Select least busy"]
```

```
    BEST --> DEAL["POST /rest/crm.deal.add<br>{title: service + date,<br>contact,
employee,<br>stage: NEW,<br>source: CHAT}"]
    DEAL --> OUT["Deal created"]
```

**TaskFlow**

```
flowchart TD
    E["create_task"] --> LLM["LLM extract from memory:<br>firstName, lastName,
<br>phoneNumber, taskTitle<br>(gemini25_flash)"]
    LLM --> CONTACT["POST /rest/crm.contact.list<br>{phone}"]
    CONTACT --> FOUND{{"Contact?"}}
    FOUND -->|"No"| SKIP_LINK["No CRM link"]
    FOUND -->|"Yes"| LINK["UF_CRM_TASK:<br>[C_contact_id]"]
    SKIP_LINK --> TASK["POST /rest/tasks.task.add<br>{title, deadline,
<br>created_by: 1,<br>responsible_id: 1}"]
    LINK --> TASK
    TASK --> OUT["result_success"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{{"Cancellation<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| DEL["POST /rest/calendar.event.delete<br>{id: booking_id}"]
    DEL --> OUT["result_success"]
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice). Major flows (availability, booking, cancellation, lead, task) have dedicated webhook test endpoints.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish and verify OAuth token exchange + working hours fetch
2. **Availability**: Ask about open slots — verify calendar events are queried and free times computed
3. **Booking**: Complete a booking — verify calendar event created in Bitrix24 with contact link
4. **Cancellation**: Cancel a booking — verify calendar event deleted
5. **Deal**: End a conversation after booking — verify deal created and assigned to least busy employee
6. **Task**: Ask agent to create a task — verify task appears in Bitrix24 with contact link

**If no action occurs:**

- Ensure OAuth credentials are set ( `bitrix_base_url` , `bitrix_client_id` , `bitrix_client_secret` )
- Verify `bitrix_access_token` was obtained
- Confirm `bitrix_owner_id` points to a valid user with a calendar
- Check feature flags are enabled for the desired operations
- Verify `bitrix_employee_position_name` matches actual employee positions (for deal assignment)

- Re-publish the project if credentials were changed

> *Note: This integration creates real records in Bitrix24. Calendar events, deals, leads, tasks, and contacts are reflected in the live CRM.*

## 5. FAQ

**Q: How does employee load balancing work for deals?** A: When a deal is created at session end, the integration fetches all active employees matching `bitrix_employee_position_name`, counts their open deals (stage=NEW), and assigns the new deal to the employee with the fewest deals.

**Q: What is the difference between deals, leads, and tasks?** A: **Deals** are auto-created when a conversation ends (if a booking was made) — they represent a business opportunity with employee assignment. **Leads** are created from external sources (e.g., Yelp) as initial contact records. **Tasks** are created on demand from conversation context with LLM-extracted details.

**Q: How is availability computed?** A: The integration queries the calendar owner's events for the date range, then generates time slots at `bitrix_duration` intervals (default: 60 min), removing slots that overlap with existing events.

**Q: Can I connect to an on-premise Bitrix24?** A: Yes. Set `bitrix_base_url` to your on-premise instance URL. The OAuth token endpoint remains at `https://oauth.bitrix.info/oauth/token/` for all Bitrix24 installations.

**Q: How does the token refresh work?** A: On HTTP 401 responses, the integration automatically refreshes the token via `https://oauth.bitrix.info/oauth/token/` with the stored refresh token. A separate HTTP connector ( `bitrix_token_connector` ) handles token operations.

**Q: What information does the LLM extract for tasks?** A: Gemini 2.5 Flash extracts `firstName`, `lastName`, `phoneNumber` (E.164 format), and `taskTitle` (max 10 words) from the conversation memory. If last name is not available, the first name is copied.

**Q: Can I disable deal creation at session end?** A: Yes. Set `bitrix_create_deal_on_conversation_end` to `False`. This disables the DealFinalizationFlow entirely.